



Winter 2026 • CSCC01

Topics Covered

- Containers
- Docker
- Writing Dockerfiles
- Docker commands
- Docker networking
- Docker volumes
- Docker Compose

Prereq3

Minimum Software Requirements

(Windows):

- Docker Desktop (if you're on window3)
- WSL

Minimum Software Requirements

(Linux):

- Docker

What is a Container?

Problem:

“Your program is crashing in production” -**Pankaj**

“But It works on my machine :p” -

Porom

“>:(” -**Pankaj**

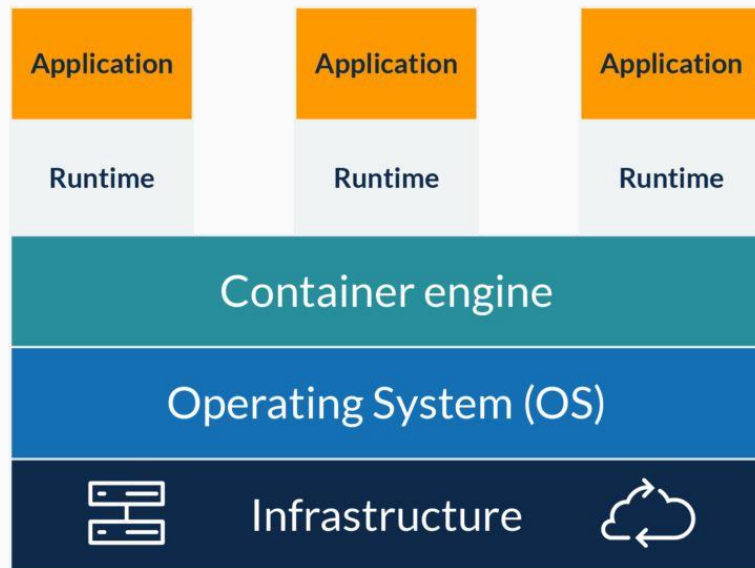
“The deployment machines are using

MySQL 6, and Node 22. We need MySQL 8, and

Node 18. Please reconfigure all the

What is a Container?

- “A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another” - **Docker’s Description of a Container**
- You can think of each container as its own computer that is running your program.

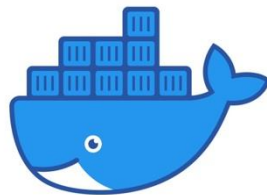


Containerization Software/Runtimes

- runc
- buildah
- LXC
- LXD
- Containerd
- **Docker ← What we're learning today**

What is a Docker?

- Docker is a Platform as a Service (PaaS) enabling users to deliver software in isolated packages called containers
- Allows you to essentially “bundle up” your code into a package (called a container) that can be deployed and run anywhere with ease
- If it runs on one machine, it'll run on the rest! (as long as it has docker installed)



What are Docker Images?

- The blueprint used to create a **container**, you can share this with people and build on top of it!
- “A Docker image is a read-only (immutable) file that contains the source code, libraries, dependencies, tools, and other files needed for an application to run inside a Docker container.”
- Image can be created by writing a **Dockerfile**
- **Containers** are created from an Image

Dockerfile3

1. Add a Dockerfile at the root directory of the project you want to dockerize
2. Dockerfiles contain a number of instructions as seen here →
3. Each Dockerfile should start with a “FROM” instruction.
4. “FROM” instructions specify which image to start from.
5. This could be an OS image (Ubuntu, Debian, Alpine), or any other image (maven, node, etc). If you select a non-OS image, make sure to verify which OS the image is running!

Instruction	Description
<code>ADD</code>	Add local or remote files and directories.
<code>ARG</code>	Use build-time variables.
<code>CMD</code>	Specify default commands.
<code>COPY</code>	Copy files and directories.
<code>ENTRYPOINT</code>	Specify default executable.
<code>ENV</code>	Set environment variables.
<code>EXPOSE</code>	Describe which ports your application is listening on.
<code>FROM</code>	Create a new build stage from a base image.
<code>HEALTHCHECK</code>	Check a container's health on startup.
<code>LABEL</code>	Add metadata to an image.
<code>MAINTAINER</code>	Specify the author of an image.
<code>ONBUILD</code>	Specify instructions for when the image is used in a build.
<code>RUN</code>	Execute build commands.
<code>SHELL</code>	Set the default shell of an image.
<code>STOPSIGNAL</code>	Specify the system call signal for exiting a container.
<code>USER</code>	Set user and group ID.
<code>VOLUME</code>	Create volume mounts.
<code>WORKDIR</code>	Change working directory.

Dockerfile3

Example: Dockerizing a Node.js Project.

```
# Start from the official Node.js 16 image
FROM node:16

* Note that the node:16 image runs on debian (the OS)

# Create app directory
WORKDIR /usr/src/app

# Copy package.json and package-lock.json to the working directory
COPY package*.json ./

# RUN npm install in our container
RUN npm install

# Copy all the source code from the current directory to the working directory in the container
COPY . .

# Set any environment variables
ENV OPENAI_API_KEY=example
ENV HUGGINGFACE_API_KEY=example
ENV STORYBOT_SERVER_URL=https://api-inference.huggingface.co/models/bigscience/bloom

# Expose the port the app runs on
EXPOSE 5000

# Run the command to start the server
CMD [ "node", "index.js" ]
```

Dockerfile3

Example: Another way to dockerize a node project (le33 preferable)

```
# Use the official Ubuntu base image
FROM ubuntu:latest
```

```
# Update the package repo, and install Node.js
RUN apt-get update \
    && apt-get upgrade \
    && apt-get install -y nodejs npm
```

```
# Create and set the working directory
WORKDIR /usr/src/app
```

```
# Copy package.json and package-lock.json
COPY package*.json ./
```

```
# Install Node.js dependencies
RUN npm install
```

```
# Copy the rest of the application code
COPY . .
```

```
# Expose the port on which your app runs
EXPOSE 3000
```

```
# Command to run your app
CMD ["npm", "start"]
```

* Start with Ubuntu image, and
install Node.js using commands
instead

How to build your image

1. Build the Dockerfile, using “docker build”

Typical usage:

docker build . -t image_name:tag

- “docker build” -the command
- “.” -The path to the dockerfile. If “.” then docker will search for “Dockerfile”. If you want to provide a path, replace with “-f file_path”
- “-t image_name:tag” provide a name, and tag for the image. E.g: CSCC01Tut:**1.0**, tag will default to “latest” if not provided.

How to start your container from an image

Run a container from the image you just created using “docker run”

Typical usage:

**docker run --name CONTAINER_NAME -d --network NETWORK_NAME --rm -p
PORT_EXTERNAL:PORT_INTERNAL image_name:tag**

- “docker run” - the command
- “--name CONTAINER_NAME” -What to name the running container. If not provided, will default to a random name.
- “-d” -detach and run the container in the background
- “--network NETWORK_NAME” -The network to run this container on ***More on this later**
- “--rm” -Remove the container, and all its associated data when it exits
- “-p PORT_EXTERNAL:PORT_INTERNAL” -Bind the port of the container to a port on the host machine. I.e. All traffic coming into PORT_EXTERNAL in the host machine, will be forwarded to the docker container's PORT_INTERNAL.
- “image_name:tag” -the name of the image to use

Docker Network3

Suppose your docker container is running a backend on port 5000.

From your machine, you can make requests to this docker container at **localhost:5000**.

Great, so **local machine** → **container** requests can be done through localhost.

But what about **container** → **container** requests?

Containers are isolated instances, they don't know anything about other running containers.

Therefore containers **can't** make requests to other containers by default. In order to make cross container requests they must be on the same **network group**.

*** NOTE: This does not apply to containerized web applications! Your containerized frontend can communicate with your containerized backend through localhost!

Docker Networks

How do we start a network group?

% docker network create GROUP_NAME

This will create a network with name: GROUP_NAME

How do containers communicate in this network?

Suppose you have two containers running, one called **backend**, and the other called **postgres** (hosted on port 5432).

backend can connect to your postgres database through <http://postgres:5432> (so CONTAINER_NAME:PORT).

Some other useful commands

docker ps:

- View all the running containers

docker stop <CONTAINER_NAME>

- Stop a container

docker logs -f <CONTAINER_NAME>

- View all the logs for the container

docker system prune --all --volumes

- remove all images, containers, volumes that are on your machine

Demo Time

- If you're on windows you need to start Docker Desktop in order to start the docker daemon.
- Other OS' should start ***docker daemon** on startup
- Download and extract the docker_demo.zip file on Quercus

* **Docker daemon** - The persistent background process which manages containers, images, volumes, etc

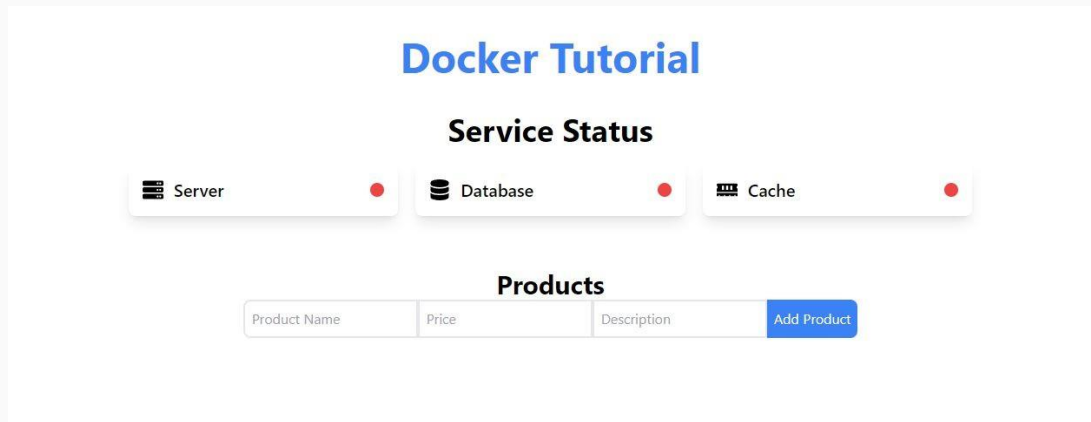
Demo Time

Setup

1. Run your frontend

% cd frontend

% npm run tutorial
2. Observe the page on localhost:3000



Demo Time

Setup Network

1. Let's setup a network to run all of our containers so they can communicate with each other. Call it "my-network".

```
% docker network create my-network
```

Demo Time

Dockerize Backend

We'll be dockerizing our stock management web application.

Let's start with dockerizing our **backend**

1. Open the project in your code editor
2. Expand the backend folder
3. Find, and complete the Dockerfile
4. Build, and run the backend image

Demo Time

Dockerize Backend Solution

~/backend/Dockerfile

```
FROM node:latest
```

```
WORKDIR /app
```

```
COPY package.json .
```

```
RUN npm install
```

```
COPY . .
```

```
CMD ["npm", "start"]
```

```
% cd backend && docker build . -t backend
```

```
% docker run --network my-network --name backend -d -m -p  
3001:3001 backend
```

Docker Tutorial

Service Status



Server



Database



Cache



Products

Product Name

Price

Description

Add Product

Tada! Our frontend is now connected to our server

Demo Time

Dockerize Database

How do we fix this?

 Database

Our backend uses the latest version of postgres. You could:

1. Go to the postgres website
2. Download the latest installer
3. Go through the setup process, and possibly install the bloatware that comes with it

Demo Time

Dockerize

Database

Or you can use Docker!

1. Run the postgres image (copy paste this into your terminal)

```
docker run -d --name postgres --network my-network --restart always -e
POSTGRES_USER=stock -e POSTGRES_PASSWORD=stock -p 5432:5432
postgres:latest
```

This will retrieve the official postgres image that is published.

- “--name postgres” -Give our container a name, doesn’t have to be “postgres”
- “--network my-network” -run the container on “my-network” so our backend container can connect it
- “-e ...” set two environment variables required by postgres (login)
- “-p 5432:5432” -Map the postgres container port 5432 to our machine’s port 5432.
- “postgres:latest” -Pull the latest postgres image

Docker Tutorial

Service Status



Server



Database



Cache



Nice! We can now start using our application. Try adding some items.

* The status update may take a minute to show up

Demo: What's left?

Cool, so we've dockerized our backend and our database.

We still have to Dockerize our frontend (Dockerfile already created for you), and redis cache that our application uses.

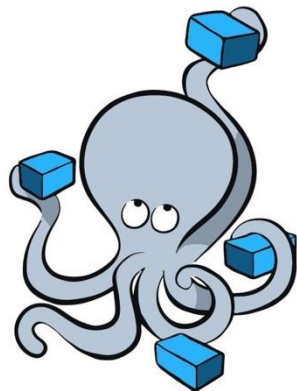
Now, every time we want to run our application, we have to run 4 “docker build”, and 4 “docker run” commands.

Is there a better way???

Yes, this is where **docker-compose** comes in.

Docker-Compose

- A tool for defining multi-container applications.
- Start by including a docker-compose.yml file in your project root directory.
 - Other names are valid but docker-compose.yml is the default
- **With docker compose we only have to run one command to start all of our containers!**



docker
Compose

Docker-Compose: Example

```
services: # Start with this tag. This will contain all the containers you want to run
  postgres: # For each container, define the block name. Can be anything, but should match the name of the container
    container_name: postgres # Name of the container
    image: postgres:latest # If you want to use published Docker image, then use this tag to specify
    restart: always # Restart the container if it stops
    environment: # Environment variables to be passed to the container
      POSTGRES_USER: stock
      POSTGRES_PASSWORD: stock
    ports: # Ports to be exposed to the host machine
      - "5432:5432"

  frontend:
    build: # Build the image from the Dockerfile
      context: ./frontend # Path to the Dockerfile
      dockerfile: Dockerfile # Name of the Dockerfile
    container_name: frontend
    ports:
      - '3000:3000'
    restart: always
```

The main docker-compose commands

docker compose up [--build] [-d] [-f FILE_PATH]

- Start our docker-compose project
- Use the **--build** flag to rebuild all the images (this is default if you haven't built the image already)
- Use the **-d** flag to run all the containers in the background (recommended)
- Use the **-f** flag to specify the file path for the docker-compose, if it doesn't follow default naming

docker compose down [-f FILE_PATH]

- Stop all the containers in our compose project
- Use the **-f** flag to specify which compose project to stop

Demo Time

Compose our project

*** Make sure to stop, and delete all your running containers before proceeding!**

Let's write a docker-compose for our project now!

One has already been created for you at the root directory. But it's missing the backend container!

Complete the missing container, and run the project!

Demo Time

Compose our project Solution

Add the highlighted section:

```
services: # Start with this tag. This will contain all the containers you want to run
  postgres: # For each container, define the block name. Can be anything, but should match the name of the container
    container_name: postgres # Name of the container
    image: postgres:latest # If you want to use published Docker image, then use this tag to specify
    restart: always # Restart the container if it stops
    environment: # Environment variables to be passed to the container
      POSTGRES_USER: stock
      POSTGRES_PASSWORD: stock
    ports: # Ports to be exposed to the host machine
      - "5432:5432"

  frontend:
    build: # Build the image from the Dockerfile
      context: ./frontend # Path to the Dockerfile
      dockerfile: Dockerfile # Name of the Dockerfile
    container_name: frontend
    ports:
      - '3000:3000'
    restart: always

  cache:
    container_name: redis
    image: redis:alpine
    restart: always
    ports:
      - '6379:6379'

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: backend
    ports:
      - "5000:5000"
    restart: always
```

Demo Time

Compose our project Solution

Run the docker-compose file

```
% docker compose up -d
```

Go to localhost:3000

Docker Tutorial

Service Status

 Server 

 Database 

 Cache 

Products

Product Name	Price	Description	Add Product
---	------------------------------------	--	--

Congrats, you have fully containerized an application!

The End

This tutorial is by no means a complete docker guide. You should also see the following:

- Docker Volumes
- Docker Container Registries
- Docker Swarm
- Multi-Stage Builds